# Assignment: Automotive Intrusion Detection - CAN Bus and Telematics Security

## Overview

Build an intrusion detection system (IDS) for automotive networks combining in-vehicle CAN bus anomaly detection with telematics behavioral monitoring. Leverage publicly available CAN bus datasets and simulated telematics data to develop ML-based detection for attacks targeting modern connected vehicles. Students must understand automotive network architectures, engineer meaningful features from time-series vehicle data, handle the unique characteristics of embedded systems telemetry, and consider deployment constraints in resource-limited automotive environments.

## Problem Statement

Modern vehicles are distributed embedded systems with dozens of Electronic Control Units (ECUs) communicating over internal networks (primarily CAN bus), connected to cloud services via telematics, and increasingly exposed to external threats. The automotive industry reports a 39% increase in cyber incidents (2023→2024), with 85% of attacks being fully remote. Students must develop detection capabilities that:

1. **CAN Bus Intrusion Detection**: Detect anomalous message patterns on in-vehicle networks that may indicate injection attacks, replay attacks, or compromised ECUs
2. **Telematics Behavioral Monitoring**: Profile normal vehicle/driver behavior and detect deviations that may indicate compromise or abuse of connected services
3. **Multi-Layer Correlation**: Combine in-vehicle and telematics signals to improve detection confidence
4. **Edge Deployment Considerations**: Design models suitable for resource-constrained automotive compute environments

# Automotive Security Context

## Why This Matters

- **Safety-Critical Systems**: Unlike IT networks, automotive attacks can affect braking, steering, and acceleration
- **Scale of Impact**: A vulnerability in telematics/OTA can affect millions of vehicles simultaneously
- **Regulatory Pressure**: UN R155/R156 and ISO/SAE 21434 mandate cybersecurity management for vehicle type approval
- **vSOC Emergence**: Vehicle Security Operations Centers need ML-driven analytics for fleet-scale monitoring

## Architecture Reference

Students should understand the following automotive subsystems (see course materials for details):

| Subsystem | Components | Security Relevance |
|---|---|---|
| **In-Vehicle Networks** | CAN/CAN-FD, LIN, Automotive Ethernet, Gateways | Lateral movement, injection attacks |
| **Telematics (V2C)** | TCU, cellular modem, OTA systems | Remote access, mass-scale attacks |
| **Infotainment (IVI)** | Android/Linux systems, apps, Wi-Fi/BT | Attack surface, pivot point |
| **ADAS** | Cameras, radar, perception, planning | Safety-critical, sensor spoofing |
| **External Interfaces** | OBD-II, mobile apps, V2X | Entry points for attackers |

# Data Sources

## Primary Datasets

- **Car Hacking Dataset (HCRL)**: Hacking and Countermeasure Research Lab
  - CAN bus traffic from real vehicle with injected attacks
  - Attack types: DoS, Fuzzing, Gear Spoofing, RPM Spoofing
  - ~4M CAN messages per attack scenario

    - Pre-labeled for supervised learning
    - Download via Dropbox link on dataset page
- **SynCAN Dataset**: Synthetic CAN data (ETAS)
    - Systematic dataset for IDS evaluation
    - Multiple attack types: flooding, suppress, plateau, continuous, playback
    - IEEE paper: "CANet: An Unsupervised Intrusion Detection System"
    - Good for reproducible benchmarking
    - Note: Repository archived but data still available
- **Survival Analysis Dataset**: CAN Intrusion Dataset (Kaggle)
    - Alternative CAN dataset with attack labels
    - Multiple vehicle scenarios

## Simulated/Derived Data for Telematics

Since real telematics data is proprietary, students should:

- **Derive telematics features** from CAN signals (speed, RPM, fuel, location proxy)
- **Generate synthetic fleet profiles** with normal/anomalous behavior patterns
- **Use public driving datasets** (e.g., comma.ai, nuScenes metadata) for behavioral baselines

## Supplementary Sources (Optional)

- MITRE ATT&CK for ICS/Automotive: Technique mappings
- SAE J3061/ISO 21434: Security engineering references
- Upstream Security Reports: Real-world incident patterns

# Required Components

## Part 1: CAN Bus Intrusion Detection (40%)

Develop an IDS for in-vehicle CAN network traffic:

1. **CAN Protocol Understanding**:
    - Parse CAN frames (ID, DLC, Data payload, timestamps)
    - Understand CAN message structure and timing characteristics
    - Document how attack types manifest in CAN traffic
2. **Feature Engineering for CAN**:
    - **Message-level features**: ID frequency, timing intervals, payload entropy
    - **Sequence features**: ID transition patterns, n-grams of message sequences

- **Statistical features**: rolling statistics, deviation from baselines
- **Signal-level features**: decoded signal values and rate-of-change (if decoding available)
- Document security rationale for each feature category

3. **Model Development**:
   - Train classifiers for attack detection (supervised: RF, XGBoost, or lightweight NN)
   - Implement anomaly detection baseline (unsupervised: Isolation Forest, One-Class SVM)
   - Compare supervised vs unsupervised approaches
   - Evaluate per-attack-type performance (DoS vs Fuzzing vs Spoofing)
4. **Temporal Analysis**:
   - CAN is time-series data—leverage temporal patterns
   - Implement windowed detection (e.g., 100ms windows)
   - Analyze detection latency requirements for safety response

## Part 2: Telematics Behavioral Monitoring (30%)

Build behavioral profiles from vehicle/driver telemetry:

1. **Feature Engineering from Telematics**:
   - **Driving behavior**: speed patterns, acceleration profiles, braking intensity
   - **Usage patterns**: trip frequency, duration, time-of-day
   - **Location patterns**: geo-clustering, route deviation (privacy-preserving features)
   - **System health**: diagnostic codes, battery/fuel patterns
2. **Behavioral Baseline Model**:
   - Train a per-vehicle or per-fleet baseline (autoencoder or statistical profile)
   - Define "normal" behavior envelope
   - Detect deviations indicating potential compromise or abuse
3. **Anomaly Scenarios**:
   - Simulate attack scenarios (unusual remote commands, credential abuse, theft patterns)
   - Evaluate detection of behavioral anomalies
   - Balance sensitivity vs false positive rate for fleet operations

## Part 3: Multi-Layer Integration (15%)

Combine CAN and telematics signals:

1. **Correlation Logic**:
   - Design how CAN-level alerts and telematics anomalies combine
   - Implement confidence scoring based on multiple signal sources
   - Consider: CAN anomaly + unusual telematics = higher confidence

2. **vSOC Alert Design**:
    - Define alert tiers (informational, suspicious, critical)
    - Map detection outputs to MITRE ATT&CK for automotive where applicable
    - Design alert format suitable for analyst triage
3. **Fleet-Scale Considerations**:
    - Discuss how to aggregate individual vehicle detections
    - Cross-vehicle correlation for systemic attacks (affecting multiple vehicles)

## Part 4: Deployment Constraints (15%)

Address automotive-specific deployment challenges:

1. **Resource Constraints**:
    - Automotive ECUs have limited compute/memory
    - Profile model inference time and memory footprint
    - Compare "heavy" models vs lightweight alternatives (e.g., decision trees, quantized NNs)
2. **Real-Time Requirements**:
    - CAN messages arrive at high frequency (thousands/second)
    - Analyze if your model can process at line rate
    - Discuss buffering, windowing, and latency trade-offs
3. **Deployment Architecture**:
    - Propose where detection should run (gateway ECU, telematics unit, cloud, hybrid)
    - Discuss pros/cons of edge vs cloud detection
    - Consider OTA update path for model updates

# Technical Requirements

## Evaluation Metrics

**CAN Intrusion Detection**:

- Per-attack-type detection rate (TPR)
- False positive rate (critical for in-vehicle—false positives may trigger warnings)
- Detection latency (time to detect attack onset)
- F1-score and confusion matrix per attack type

**Telematics Anomaly Detection**:

- Anomaly detection precision/recall

- False positive rate per vehicle-day
- Time-to-detect for simulated attack scenarios

**Deployment Metrics**:

- Model inference time (ms per window)
- Memory footprint (KB/MB)
- Throughput (messages/second sustainable)

## Validation Methodology

- **Temporal ordering**: Train on earlier data, test on later data (many CAN datasets have timestamps)
- **Attack holdout**: Consider holding out one attack type to test generalization
- **Per-vehicle split**: For telematics, ensure train/test separation by vehicle ID
- **Cross-validation** with appropriate splits for time-series

# Deliverables

1. **Jupyter Notebook(s)** with complete implementation:
    - CAN data exploration and attack analysis
    - Feature engineering pipeline with documentation
    - Model training and comparison
    - Telematics behavioral modeling
    - Integration and alert design
    - Deployment analysis
    - Well-documented code with markdown explanations
2. **Trained Models**:
    - CAN intrusion detection model(s)
    - Telematics behavioral baseline model
    - Serialized in deployment-friendly format
3. **Technical Report** (3-5 pages) covering:
    - **Automotive Context**: Why intrusion detection matters for vehicles
    - **CAN Analysis**: Attack characteristics and feature rationale
    - **Telematics Approach**: Behavioral modeling design
    - **Results**: Detection performance by attack type
    - **Integration**: Multi-layer correlation strategy
    - **Deployment**: Resource analysis and architecture recommendation
    - **Limitations**: Known gaps, real-world deployment challenges
4. **vSOC Alert Specification** (1 page):

- Alert schema (fields, severity levels, supporting data)
- Mapping to automotive threat taxonomy
- Recommended analyst response workflow

# Evaluation Rubric

## Research & Domain Understanding (15%)

- Understands automotive network architecture (CAN, ECUs, domains)
- Explains attack types with technical accuracy
- Connects approach to automotive security standards (ISO 21434, UN R155)
- Discusses safety implications of detection failures

## Technical Implementation (45%)

- **CAN IDS (25%)**:
  - Thorough feature engineering from CAN traffic
  - Multiple models compared with proper validation
  - Per-attack performance analysis
  - Temporal aspects handled correctly
- **Telematics Behavioral Detection (15%)**:
  - Meaningful behavioral features extracted
  - Baseline model trained and evaluated
  - Anomaly scenarios analyzed
- **Code Quality (5%)**:
  - Clean, reproducible code
  - Efficient data handling (CAN datasets can be large)
  - Well-documented functions

## Integration & Alert Design (20%)

- Logical multi-layer correlation approach
- Alert taxonomy aligned to operational needs
- MITRE mapping where applicable
- Fleet-scale considerations addressed

## Deployment Analysis (15%)

- Resource profiling completed
- Realistic assessment of real-time feasibility
- Architecture recommendation justified
- Trade-offs clearly articulated

## Documentation (5%)

- Clear technical report
- Reproducible notebook
- Honest limitation discussion
- Professional presentation

# Learning Outcomes

Upon completing this assignment, students will:

1. **Understand Automotive Architectures**: Grasp how in-vehicle networks, telematics, and cloud systems interconnect and where security boundaries exist
2. **Work with Time-Series Network Data**: Engineer features from high-frequency, low-level network traffic with strict timing characteristics
3. **Apply ML to Embedded Systems Security**: Adapt ML techniques for domains with resource constraints and safety implications
4. **Design Detection for Safety-Critical Systems**: Balance detection sensitivity against false positive costs that could affect driver experience or safety responses
5. **Think Fleet-Scale**: Consider how individual vehicle detections aggregate and correlate across vehicle populations
6. **Connect to Regulatory Context**: Understand how automotive cybersecurity regulations shape detection and response requirements

# Recommended Tools & Libraries

- **Data Processing**: `pandas`, `numpy`, `scikit-learn`
- **CAN Parsing**: `cantools`, `python-can` (for understanding, not live capture)
- **Machine Learning**: `scikit-learn`, `xgboost`, `lightgbm`
- **Deep Learning (optional)**: `tensorflow-lite`, `onnx` (for deployment analysis)
- **Time-Series**: `tsfresh`, `tslearn`

- **Visualization**: `matplotlib`, `seaborn`, `plotly`
- **Profiling**: `memory_profiler`, `timeit`

# Optional Extensions (Bonus Points)

1. **Deep Learning for CAN**: Implement LSTM/1D-CNN for sequence-based detection
2. **Transfer Learning**: Train on one vehicle, test generalization to another
3. **Adversarial Robustness**: Test if attackers can craft CAN messages that evade detection
4. **V2X Security**: Extend to V2V/V2I message anomaly detection
5. **EV Charging Security**: Apply similar techniques to charging protocol (OCPP) data
6. **Edge Deployment**: Actually deploy model to Raspberry Pi or similar and measure performance
7. **Explainability**: Use SHAP to explain which CAN features triggered alerts

# Resources

## Datasets

- Car-Hacking Dataset (HCRL)
- SynCAN Dataset (ETAS)
- comma.ai Driving Dataset (for behavioral baselines, includes raw CAN)

## Technical References

- CAN Specification (Bosch) - Protocol details
- ISO/SAE 21434 - Automotive cybersecurity standard
- UN R155/R156 - Regulatory requirements
- MITRE ATT&CK for ICS - Technique taxonomy

## Research Papers

- Miller & Valasek (2015) - "Remote Exploitation of an Unaltered Passenger Vehicle" - Seminal automotive hacking research
- Song et al. (2020) - "In-Vehicle Network Intrusion Detection Using Deep Learning" - ML approaches survey
- Upstream Security (2025) - "Global Automotive Cybersecurity Report" - Industry threat landscape

## Course Materials

- Automotive Security Background Notes (AutomotiveSecurity/Background.md)
- Automotive Attack Research (AutomotiveSecurity/Research.md)

# Submission Guidelines

- Submit all deliverables via course platform
- Jupyter notebooks must be executable (include `requirements.txt` )
- Include README with setup instructions and data download steps
- Models serialized in standard format (pickle, joblib, or ONNX)
- Technical report as PDF (3-5 pages, excluding appendices)
- Large data files should be referenced (not uploaded)—provide download instructions

# Frequently Asked Questions (FAQ)

## Data & Setup

**Q: The CAN datasets are very large. How should I handle them?**

A: Options:

1. Sample a subset for development (e.g., first 1M messages per scenario)
2. Use chunked reading ( `pd.read_csv(chunksize=...)` ) for memory efficiency
3. Pre-process and save feature-engineered versions as intermediate files
4. Document your sampling approach

**Q: I don't understand CAN message format. Where can I learn?**

A:

- Each CAN message has: Arbitration ID (11-bit), DLC (length), Data (0-8 bytes), Timestamp
- The ID identifies the message type/source ECU
- The Data payload contains encoded signals (speed, RPM, etc.)
- Start with the dataset documentation and CAN protocol tutorials online
- You don't need to fully decode signals—many features work on raw ID/timing patterns

**Q: How do I simulate telematics data if I only have CAN data?**

A: Derive telematics-like features from CAN:

- Extract speed/RPM signals (often labeled in datasets or can use ID patterns)
- Aggregate into "trip" summaries (duration, distance proxy, speed profile)
- Add synthetic location features (random clusters representing routes)
- The goal is demonstrating behavioral modeling—simulation is acceptable

**Q: Can I use a different CAN dataset?**

A: Yes, alternatives like ROAD dataset, CrySyS dataset, or others are acceptable. Just ensure:

- Labeled attack scenarios for supervised evaluation
- Sufficient volume for meaningful analysis
- Document why you chose an alternative

# Technical Implementation

**Q: What window size should I use for CAN detection?**

A: Common approaches:

- 50-200ms windows for near-real-time detection
- Fixed message count windows (e.g., 100 messages)
- Experiment with multiple window sizes and report trade-offs
- Smaller windows = faster detection but noisier; larger = more stable but slower

**Q: My model has very high accuracy (>99%). Is that expected?**

A: Yes, but be careful:

- CAN attacks are often "obvious" in retrospect (massive injection bursts)
- Check per-attack-type performance—some attacks are harder than others
- Test on held-out time periods and attack types
- Focus on detection latency and false positive rate, not just accuracy

**Q: Should I decode CAN signals or work with raw bytes?**

A: Both approaches are valid:

- **Raw/ID-based**: Works without decoding, more generalizable
- **Signal-based**: If you have DBC files or known signal mapping, decoded features may be more interpretable
- Most research works effectively with ID-based and raw payload entropy features

**Q: How do I handle the class imbalance (mostly normal, few attack messages)?**

A: Options:

- Window-based labeling (label window as attack if contains any attack messages)
- Undersample normal or oversample attacks
- Use appropriate metrics (F1, precision-recall, not just accuracy)
- Anomaly detection approaches naturally handle this

# Deployment Analysis

**Q: I don't have an embedded system to test on. How do I analyze deployment?**

A: Simulate constraints:

- Profile inference time on your laptop ( `timeit` )
- Measure model memory footprint ( `pickle` size, parameter count)
- Compare: scikit-learn RF vs XGBoost vs simple decision tree vs small NN
- Research typical automotive ECU specs (e.g., ARM Cortex, 256MB RAM) and estimate feasibility

**Q: What's the target inference time for real-time detection?**

A: Back-of-envelope:

- CAN bus runs at 500 kbps–1 Mbps, ~3000-8000 messages/second
- If using 100-message windows, you need inference in <20-30ms
- For this assignment, demonstrate awareness of constraints; you don't need to meet them precisely

**Q: Edge vs cloud detection—what should I recommend?**

A: Discuss trade-offs:

- **Edge (in-vehicle)**: Low latency, works offline, limited compute, harder to update
- **Cloud**: More compute, easier updates, requires connectivity, higher latency
- **Hybrid**: Common recommendation—fast edge detection + cloud correlation and learning
- There's no single right answer; justify your recommendation

# Integration & Alerts

**Q: What MITRE techniques apply to automotive?**

A: Look at MITRE ATT&CK for ICS, plus automotive-specific mappings:

- T0855: Unauthorized Command Message
- T0843: Replay Attack
- T0830: Adversary-in-the-Middle
- Map your attack types (DoS, Spoofing, Fuzzing) to relevant techniques

**Q: What should the vSOC alert contain?**

A: Include:

- Timestamp, Vehicle ID, Alert Severity
- Detection source (CAN IDS, Telematics Behavioral)
- Attack type classification and confidence
- Supporting evidence (anomalous CAN IDs, feature deviations)
- Recommended action (monitor, investigate, isolate)

## Scope & Effort

**Q: What's the expected time commitment?**

A: Plan for 15-25 hours:

- Data exploration & CAN understanding: 3-5 hours
- CAN feature engineering & modeling: 5-7 hours
- Telematics behavioral modeling: 3-5 hours
- Integration & deployment analysis: 3-4 hours
- Documentation & report: 2-4 hours

The CAN dataset exploration phase often takes longer than expected—start early.

**Q: Can I focus more on CAN and less on telematics?**

A: The CAN component (40%) is weighted higher. If you do excellent CAN work but simpler telematics behavioral modeling, that's acceptable. However, demonstrating multi-layer thinking is important for the integration component.

**Q: Can I use existing CAN IDS implementations as reference?**

A: Yes, you can reference published research and GitHub implementations, but:

- Cite everything you reference
- Don't copy-paste code without understanding
- Your feature engineering and analysis must be your own

- TAs may ask you to explain any approach you use

**Safety Note**: This assignment uses pre-recorded CAN data. Never connect to a real vehicle's CAN bus without proper authorization and safety precautions. Injecting messages into a live vehicle network is dangerous and illegal without explicit permission.

**Academic Integrity**: You may discuss approaches with classmates, but all code and analysis must be your own work. Properly cite external code, libraries, or research papers used. LLM use for code generation must be disclosed and validated.

**NIST CSF Mapping**: DETECT → Security Continuous Monitoring, Anomalies and Events
**MITRE ATT&CK Coverage**: T0855 (Command Message), T0843 (Replay), T0830 (MITM)
**Automotive Standards**: ISO/SAE 21434, UN R155/R156